Software Engineering for AI Systems: Lessons from Building and Evaluating Obfuscation Detectors

Hantang Zhang, Umeå University

WASP Software Engineering Course Module 2026

1 Introduction

My PhD research is in Android software security. I work on *software obfuscation detection*, the topic of this essay. It is the first of three planned stages of my PhD: (1) build an obfuscation detector; (2) use it to locate obfuscated regions and remove them with large language models (LLMs); (3) apply both to malware detection and analysis.

Obfuscation rewrites a program so that its behaviour stays the same but its structure becomes hard to read for humans and tools. I focus on *control-flow obfuscation* at the smali level (readable Dalvik bytecode), and on two techniques: *opaque predicates*, where a condition looks variable but is in fact constant, so one branch is dead code; and *control-flow flattening*, where sequential blocks are replaced by a dispatcher and a set of `case` blocks. Malware authors use both to hide behaviour, so detecting and removing them is a prerequisite for stage three.

The project started as a machine-learning project and turned into an evaluation project. I built three generations of detectors: XGBoost over 28 control-flow-graph features; CodeBERT with domain-adaptive pre-training on smali; and a generative LLM used zero-shot (a locally deployed Qwen3-14B plus five commercial API models). For training and testing I obfuscated clean APKs with three real third-party tools and with my own controllable obfuscator, `cfobf`, which implements 51 opaque-predicate templates plus parameterised flattening and records metadata for every injection.

The first two generations reached 0.96 to 0.99 on their own benchmarks. But every obfuscation tool leaves a fingerprint: fixed templates, opcode sequences and label names. A classifier can score well by memorising fingerprints without understanding that a branch is constant. In real samples the structural and the semantic signal always co-occur, so a standard test set cannot tell them apart. I learned this the hard way: several good-looking results came from leaked jump-label prefixes, from opcode habits of my own obfuscator, and from samples a third-party tool had silently left unobfuscated but still labelled positive.

To separate the two signals I constructed a paired test set, *Test G*. The obfuscator injects the same template into a method twice: once as a real opaque predicate, and once as a "twin" whose two constants are swapped so that the condition depends on the input. Both sides have identical opcode and token multisets; only the semantics differ. On Test G the discrimination score (recall on the obfuscated side minus false-positive rate on the twin side) is 0.0004 for XGBoost, 0.0014 for CodeBERT and 0.81 for the LLM. Switching off the LLM's reasoning drops its paired accuracy from 0.81 to chance level while barely moving the other test sets. The older detectors recognise tools; the LLM, at least in part, reads the code.

Models that score well for the wrong reasons, silent bugs in the data pipeline, test sets contaminated by shortcuts, and conclusions I had to retract: this is the experience I bring to the course, and the lens for the rest of this essay.

2 Lecture principles

2.1 Invariants and metamorphic testing

The quality-assurance lecture presented metamorphic testing as a way around the oracle problem: instead of knowing the correct output for one input, we state a relation that must hold between the outputs of two related inputs. All the ML examples were invariance relations, such as a credit model that must give the same answer when only the gender field changes [4].

Test G is a metamorphic test, although I did not call it that when I built it. My starting point was that the surest way to remove the structural signal is to make the structure identical, so I inject the same template into the same method twice and swap the two constants. Both sides have identical opcode and token multisets; only the semantics differ. What I had in mind was an invariant, but the relation runs in the opposite direction to the lecture examples: swapping the constants must *flip* the label. For obfuscation detection I think this "must change" relation is the more useful one, because the pair changes the meaning of the code rather than its surface, and a model that ignores the change is exactly the model that has learned fingerprints. As the lecture recommends, the oracle is generated: my symbolic range evaluator proves which side is constant.

Seen this way, Test G is one relation out of a family. The obvious invariance relation to add is register renaming: rename the registers, keep the semantics, and the prediction should not move. I already normalise register numbers for CodeBERT at preprocessing time, but I have not used renaming as a test. One concern remains. If I fine-tune on Test G pairs, the relation itself may turn into a shortcut: the model could learn to look only at the local arithmetic around the predicate and handle only this one kind of obfuscation, losing generality. This is why my fine-tuning plan treats "Test A/B/C rise, Test G falls" as evidence of an amplified shortcut rather than as progress.

2.2 Confirmation bias

The behavioural software engineering lecture [4] listed confirmation bias among the cognitive biases that survive even when we know about them. We look for evidence that supports what we already believe; in software work this shows up as running more tests that show the system works than tests that try to break it. The mitigations mentioned were traceability, deliberately looking for disconfirming evidence, and generating several alternative solutions instead of one.

My project is a case study of this bias. Some obfuscation tools skip a method silently when its structure does not meet their preconditions: the exit code is normal, the output APK is produced, and nothing is injected. For dProtect this affected 71.5% of the "obfuscated" samples in one dataset. The models still separated these samples from clean ones, because the format round trip left traces of its own, so every metric looked fine and the problem stayed hidden for months. I found it only because the results stayed almost the same no matter how I changed the model, which eventually made me suspect the data. The disconfirming evidence arrived by accident, not because I went looking for it. The pattern repeated with two other results that I later withdrew.

The practices I use now match the lecture's mitigations almost one to one: every number in my reports carries an anchor to the experiment record it came from, the three possible outcomes of the LoRA experiment have their interpretation written down before the run, and the report keeps a page listing what we cannot claim. None of this existed at the start; the hard check that now rejects a sample unless it adds at least one new branch structure was written after the contamination was found. Of the three mitigations, generating multiple alternatives is the one I find hardest in a one-person PhD project. My own design usually looks good to me, and a good result is convincing even when it should not be. A validation set that measures the real capability is the way out, but it can carry its own bias: Test F matches clean and obfuscated methods on only three dimensions with weights I chose by hand, so a shortcut on a fourth dimension would pass through unnoticed.

3 Guest-lecture principles

3.1 From a goal to a measurable requirement, and what must not happen

Julian Frattini's requirements engineering lecture [6] made two points that describe my project better than I would like. First, statements live on different levels of abstraction. "The system shall be secure" is a goal in the context layer; it has to be refined into something checkable, either by decomposing the goal into measurable quality requirements or by writing a misuse-case that states what must not happen. Second, the order of work matters: understand the problem before building the solution, because a defect costs roughly ten times more for every phase it survives.

My project ran in the opposite order. The initial requirement was a solution-space statement: train an obfuscation detector with high accuracy. The models satisfied it quickly on an in-distribution test set. Only through the experiments did it become clear that the real need was a comprehensive validation of what the model can do, which then extended naturally into the requirement that the detector must rely on semantics rather than on tool fingerprints. In the lecture's terms, the "what" was discovered by building the "how", and the months of experiments on contaminated data are the price of that order.

Written as a refinement chain, the project now reads: goal, detect control-flow obfuscation; requirement, the detector shall read the code and not recognise the tool; measurable requirement, discrimination on Test G, defined as recall on the obfuscated side minus the false-positive rate on the twin side. I do not see a gap in the metric itself. What the lecture made me reconsider is the weight of the misuse-case side. The twin pairs and Test F are both "shall not" requirements: the detector shall not flag a normal condition, and shall not flag clean code that merely looks obfuscated. The discrimination metric weights them equally with the positive requirement. But the context layer of my project is software security, where the purpose is to surface suspicious fragments for further analysis, and an occasional false positive on clean code does little harm there. So the negative requirements can be somewhat looser than the positive one, and the threshold can reflect that. This is a decision that belongs to the "why" layer, and I could only make it once the requirement was explicit.

3.2 Turning an obscure, atomic AI component into something verifiable

The RE4AI part of the lecture described data-driven components as needing no formal problem statement, generally obscure, and generally atomic, in contrast with traditional software that is formal, transparent and decomposable. AI is a solution, so it does not change requirements engineering itself, but an AI component implies requirements at its own level of abstraction. Ferrari and Spoletini's roadmap, shown at the end of the lecture, adds the complement: LLMs make formal methods usable, and formal methods make LLM-based work trustworthy.

My next design step is an instance of that complement. The error analysis of the LLM detector showed that it locates the relevant condition with accuracy 1.0 and never misses a real opaque predicate; all its errors come from deciding that a variable condition is constant. I also observed that only very large models are good at this kind of numerical reasoning, while my symbolic range evaluator, which already generates the labels, can prove constancy at almost no computing cost. So the plan is to let the LLM do the extraction only (find the condition, trace the registers, emit the constraint as structured output) and hand the decision to the evaluator. The atomic component becomes two components with a checkable interface, and the output is a constraint that anyone can re-verify.

The costs are real. The evaluator has to be maintained, the architecture gets more complex, and the evaluator only helps for opaque predicates; for flattening and for non-arithmetic predicate families the system still depends on the LLM alone. The LLM also brings requirements that the XGBoost and CodeBERT versions never had. For local deployment the binding constraint is GPU memory, and a measured self-consistency rate of 0.86 means that reproducibility has to be tested rather than assumed. These are requirements at the level of the component, as the lecture predicted.

4 Data scientists, software engineers, and AI engineers

The chapters [1] draw the line between the two roles by training and habit. Data scientists come from statistics and machine learning, work exploratively in notebooks, and judge their work by accuracy on held-out data, with little attention to latency or cost. Software engineers deliver a product that meets user needs within budget and time, which means requirements, testing, deployment and constant trade-offs between qualities. The authors admit the distinction is oversimplified but say it matches practice. I largely agree, with one addition. The difference is not only what each role cares about, metrics versus working functionality. A good data scientist needs some software engineering basis to know whether their own metric deserves trust, and to build the model they actually need for their question. My project is an example: the three detectors were data-science work, while the pitfalls (leaked labels, silently skipped samples, no checks on the data pipeline) were software engineering pitfalls. When one person plays both roles, nobody's job is to distrust the pipeline, which is why the problems stayed hidden for so long.

The system-wide view applies directly. My detector is one component in a chain (detection, then deobfuscation, then malware analysis), and the system goal is to help an analyst find hidden behaviour, not to maximise balanced accuracy. Three consequences follow. The metric should reflect the downstream use: a false positive costs an analyst some time, a miss hides code, so the two error types should not carry equal weight. The model is an unreliable function that needs non-ML safeguards, and the symbolic evaluator plays the role of the thermal fuse in the chapters' toaster example. And the data pipeline is part of the system: the decision to extract every sample with one tool changed what the models learned more than any hyperparameter did.

I do not think the two roles should stay separate; each should learn from the other. The realistic form of that in a one-person PhD project is not to learn everything, but to borrow a small set of engineering practices into the research workflow: data-level checks, pre-registered interpretations of results, and versioned test sets. This is the chapters' T-shaped team member: depth in one role, enough breadth in the other to check the contract between them.

Where does AI engineering fit? Training a model has become the easy part. What decides whether an AI tool can be trusted is the design of the whole process around it, and above all the design of an evaluation that can be believed. Test G is engineering work; it is neither data science in the chapters' sense nor MLOps. So I place AI engineering as software engineering for AI systems plus a discipline of its own, the construction of trustworthy evaluations. The chapters note that the term is often used narrowly for pipelines and deployment; I think that is the smaller part.

Foundation models have already moved the boundary in my own work. With the six LLMs I trained nothing; the work became prompt design, evaluation design and infrastructure (GPU memory, API cost, self-consistency). The model is an external unreliable component behind an API, and "model work" has turned into specification and test work. AI coding assistants write pipeline code quickly, but the failures I hit were silent ones that an assistant does not report, so review and tests matter more rather than less. My planned extraction-plus-evaluator pipeline is a small agentic workflow, and its design question is where to draw the tool boundary. Writing models is no longer the main skill; using models, evaluating their results, and designing the review and tests around them is.

5 Paper analysis

5.1 Paper 1: Vonderhaar, Elvira and Ochoa, "Generating and Verifying Synthetic Datasets with Requirements Engineering" (CAIN 2025) [2]

Core idea and SE importance. The paper treats a synthetic training set as a software product that has to be verified against requirements rather than inspected ad hoc. Requirements are written as statements about the downstream model ("the object detector shall detect a toaster when it is in the left part of the image"), turned into prompts for a diffusion model, and every generated image is checked twice: a human compares it with the prompt, and a YOLOv8 model stands in for the downstream detector. Each image passes or fails, a category is accepted when at least half of its images pass, and the chain from specification to prompt to image to verdict

is traceable. The SE value is that data generation moves from "generate and hope" to a requirements-driven process with an explicit acceptance criterion and a trace. The authors note that their specifications cover only object, count and position, and that the manual check is subjective.

Relation to my research. `cfobf` is a synthetic data generator too, and the paper made me see that it already contains four requirements that were never written down: (1) structural increment, after injection the method has at least one more conditional, switch or back-edge than before; (2) constancy, the injected predicate is constant over the full range of its operands, and the twin has both branches reachable; (3) matched negatives, clean samples match obfuscated ones in length, branch count and arithmetic density; (4) no shortcut, obfuscated samples cannot be separated by six trivial probes. Two requirements were missing. Samples from third-party tools were never required to prove that something was actually injected, which is exactly how the silently skipped samples entered the positive class. And "assembles and parses in the downstream pipeline" was never an explicit requirement, although it is the analogue of the paper's "YOLO can detect it".

On the verification step I disagree in one place. A model of the same family as the downstream model is not safe as the only verifier, because it shares the downstream model's blind spots. My own project shows why: CodeBERT nearly passed on fingerprints, and a canary check nearly passed for the wrong reason as well. The verifier should be independent of, and of a different kind from, what it verifies; my constancy evaluator is a program-analysis tool, so it cannot learn a fingerprint. The verifier also needs its own negative test, an input built to fail, proving that the gate can reject something. Otherwise it is a gate that is written but never read.

A larger project. Consider a pre-publication review pipeline for an app store. Every submitted APK is assessed automatically: is it obfuscated, how heavily, and does the obfuscation hide malicious control flow? The output is a risk score with evidence for a human reviewer; the users are the store's reviewers and the developers who appeal a rejection. AI sits on two sides of the pipeline. On the generation side this paper applies directly: requirement-driven synthetic obfuscation samples from `cfobf` feed training and regression data, and every batch passes a verification gate before use. On the detection side a hybrid detector (Paper 2) does the classification. My WASP research is the control-flow obfuscation stage of the detection side, and Test G pairs serve as the regression gate for every model update.

Adapting my research. Three long-term changes. Collect the implicit requirements now scattered through the code into one traceable document, from requirement to generator parameter to sample to verdict, in the style of my unified evaluation design. Turn verification from a one-off script into a blocking gate on every build, backed by an independent second implementation of the ground-truth labeller, which my `verify_gt` already is. And treat requirement coverage as a first-class metric: which obfuscation mechanisms, value ranges and injection sites the requirements cover and which they do not, instead of counting samples.

5.2 Paper 2: Ding et al., "Vulnerability Detection with Code Language Models: How Far Are We?" (ICSE 2025) [3]

Core idea and SE importance. The paper audits the datasets and metrics used to evaluate code language models for vulnerability detection and finds them unfit. Automatic labelling marks every function touched by a security fix as vulnerable, giving label accuracy between 24% and 60%; up to 18.9% of test samples are exact copies of training samples; random splits let models train on the future and test on the past; and accuracy and F1 hide the trade-off between missed vulnerabilities and false alarms that matters in security. PrimeVul repairs the data (de-duplication and two stricter labelling rules) and the evaluation (chronological splits; VD-S, the false-negative rate at a false-positive rate below 0.5%; and pair-wise evaluation on vulnerable/patched pairs with at least 80% textual overlap, reported as four outcomes P-C, P-V, P-B and P-R). Under this evaluation a model that scored 68% F1 on BigVul scores 3%, and GPT-4 with chain-of-thought is no better than random on pairs. For AI engineering the lesson is that the benchmark is part of the system, and it can be the component that is broken.

Relation to my research. Almost every finding has a twin in my project: their label noise is my silently skipped samples labelled positive; their duplicates are my leaked label prefixes; their pair-wise evaluation is Test G; their "models rely on textual patterns" is my fingerprints; their call for hybrid systems with program analysis is my extraction-plus-evaluator plan. Two differences matter. Test G is stricter, since its synthetic

pairs have identical token multisets; PrimeVul is more realistic, since its pairs come from real patches. And I will adopt their four-way reporting: P-V (twin also flagged) and P-B (neither flagged) separate "false positive on the twin" from "missed entirely", which my current reporting collapses. My XGBoost result, 100% ties on Test G, is exactly this story: value-blind features must give the same answer on both sides.

On the metric I would turn VD-S around. It is designed for a setting where false alarms are expensive. In obfuscation detection a false positive on a clean method costs a second look, while a miss hides code, so I would fix a high recall and report the false-positive rate under it, and keep that rate per population (twin, flattening, natural negatives) rather than pooled, because the cost of a false positive depends on which negatives it lands on. Their GPT-4 result against my 0.81 discrimination is explained by the task, not the model: their pairs often need cross-function context that even program analysis cannot supply, while my Test G predicates are decidable inside the method by enumerating a value range. The 0.81 says more for "decompose the problem into decidable sub-steps", which is their own recommendation, than for LLM strength in general; the price is that Test G is a diagnostic track, not the wild.

The larger project and my role. In the app-store pipeline above, this paper shapes the detection side: a hybrid where the program-analysis evaluator decides the provable part and the LLM or CodeBERT covers the unprovable semantics, scored in VD-S style under a recall constraint, with per-population false-positive rates so that errors on one class of negatives cannot hide in a total. My detector is the control-flow obfuscation stage, and the pairing idea becomes the regression gate.

Adapting my research. Two long-term changes. First, make the evaluator's capability boundary an evaluation object in its own right: my oracle already holds 129 cases that program analysis cannot decide, and those are the only cases where an LLM can add something beyond the provable, so "does the LLM exceed the provable range" deserves its own test. Second, institutionalise decomposition: never let the LLM judge a whole method in one step, but locate the conditions first, then judge constancy condition by condition, with each step cross-checked by the evaluator and the evaluator's verdict winning any conflict.

6 Evidence, originality, and GenAI-use transparency

The main sources I relied on, and what kind of source each is:

- Kästner, *Machine Learning in Production*, chapters 1 and 2 [1]: book chapters (MIT Press, 2025; open web edition).
- Vonderhaar, Elvira and Ochoa, CAIN 2025 [2]: peer-reviewed conference paper.
- Ding et al., ICSE 2025 [3]: peer-reviewed conference paper.
- Feldt, course lectures 02 and 03 [4]; Lenberg [5] and Frattini [6] guest lectures: lecture slides, i.e. course material.
- Ferrari and Spoletini (2025), cited only through Frattini's slides; I have not read the original.
- My own project documents and experiment records: primary material.

The sources I trust most are the two peer-reviewed papers, because I reached them through Google Scholar, a real retrieval path that has been tested over time, and read them in full. I trust the lecture slides as a record of what was taught rather than as evidence on their own, and the second-hand reference to Ferrari and Spoletini least.

I used Claude to scan the CAIN 2025 accepted-paper list and propose candidates close to my project, after which I chose the two papers and downloaded them via Google Scholar and IEEE Xplore. I also used Claude to polish the whole assignment: it corrected my English expression and grammar so that the text reads correctly, and it did some translation work, since English is not my native language.

References

[1] C. Kästner, *Machine Learning in Production: From Models to Products*, MIT Press, 2025. Chapters 1 and 2, open edition: https://mlip-cmu.github.io/book/01-introduction.html and https://mlip-cmu.github.io/book/02-from-models-to-systems.html

[2] L. Vonderhaar, T. Elvira and O. Ochoa, "Generating and Verifying Synthetic Datasets with Requirements Engineering," in *Proc. IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN)*, 2025, pp. 212–221. DOI: 10.1109/CAIN66642.2025.00032

[3] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray and Y. Chen, "Vulnerability Detection with Code Language Models: How Far Are We?," in *Proc. IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025, pp. 1729–1741. DOI: 10.1109/ICSE55347.2025.00038

[4] R. Feldt, "02 – Quality Assurance and Testing in SE, QA for ML, ML for QA" and "03 – Behavioral Software Engineering," lecture slides, WASP Software Engineering Course Module, 2026.

[5] P. Lenberg, "Importance of Human Aspects in Software Engineering," guest lecture slides, WASP Software Engineering Course Module, 2 June 2026.

[6] J. Frattini (with additions by R. Feldt), "Requirements Engineering: A Brief Introduction" and "Requirements Engineering for Artificial Intelligence-based Systems," guest lecture slides, WASP Software Engineering Course Module, 1 June 2026.